UPIIC

-Unidad Profesional Interdisciplinaria de Ingeniería Campus Coahuila-

IPN

Instituto Politécnico Nacional

Asignatura: aprendizaje de maquina

Alumno: Emmanuel Álvarez Cañedo

Carrera: Ingeniería en Inteligencia Artificial

teoría del aprendizaje computacional

Los temas de la teoría del aprendizaje computacional comparten un interés común en el desarrollo de algoritmos que pueden aprender a partir de datos. Estos algoritmos se utilizan para una amplia gama de tareas, como la clasificación, la regresión y la detección de anomalías.

En la teoría computacional se puede entender de mejor manera como el funcionamiento de la mente humana que sea similar a la de una computadora, pero para que una computadora funcione como humano necesitas de datos para analizar mientras el ser vivo puede aprender de datos, pero de forma más empírica

El problema de la dimensionalidad.

Primero que nada hay que saber ¿Qué es la dimensionalidad? Para luego poder encontrar el detalle de la teoría pensemos que hay una variable observable es fácil manipula una variable obviamente sin embargo que pasaría si hay más de 10 variables observable se vuelve más difícil analizar 10 variable el problema es que son variables predictivas que salen es como decir tengo una manzana verde solo hay dos variables color y fruta pero que pasa si cada manzana verde es de diferente especie eso quieres decir que hay una variable predictiva ya se vuelven más variables como fruta, color, especia a y especie b lógicamente el problema radica en lo siguiente son tantas variable que se vuelve difícil hacer operación porque son demasiadas variables así que la solución para esto es que se puede usar un algoritmo de reducción dimensionalidad para el problema y hacer una mejor predicción.

Linealidad vs No linealidad

En el aprendizaje automático, la linealidad vs no linealidad se refiere a la relación entre las características y la salida. Los algoritmos lineales se basan en la suposición de que la relación es lineal, mientras que los algoritmos no lineales pueden capturar relaciones más complejas.

Linealidad

Una relación lineal es aquella en la que un cambio en la entrada produce un cambio proporcional en la salida. Por ejemplo, la relación entre la distancia recorrida y el tiempo es lineal. Si duplicamos la distancia, también duplicaremos el tiempo.

Los algoritmos lineales se basan en la suposición de que la relación entre las características y la salida es lineal. Estos algoritmos son a menudo más fáciles de entender y de implementar que los algoritmos no lineales. Sin embargo, los algoritmos lineales pueden no ser adecuados para tareas que requieren capturar relaciones complejas.

Algunos ejemplos de algoritmos lineales:

Regresión lineal

No linealidad

Una relación no lineal es aquella en la que un cambio en la entrada no produce un cambio proporcional en la salida. Por ejemplo, la relación entre el crecimiento de un organismo y la temperatura es no lineal. A medida que la temperatura aumenta, el crecimiento del organismo aumenta a un ritmo cada vez mayor.

Los algoritmos no lineales pueden capturar relaciones más complejas que los algoritmos lineales. Estos algoritmos son a menudo más precisos para tareas que requieren capturar relaciones complejas. Sin embargo, los algoritmos no lineales pueden ser más difíciles de entender y de implementar que los algoritmos lineales.

Algunos ejemplos de algoritmos no lineales:

Regresión logística no lineal

Árbol de decisión

Red neuronal artificial

Elección de algoritmos

La elección de un algoritmo lineal o no lineal depende de la tarea específica en cuestión. Si la relación entre las características y la salida es lineal, entonces un algoritmo lineal puede ser suficiente. Sin embargo, si la relación es no lineal, entonces un algoritmo no lineal puede ser necesario para obtener mejores resultados.

Ventajas y desventajas

Algoritmos lineales

Ventajas:

Fáciles de entender y de implementar

Menos propensos al sobreajuste

Desventajas:

Pueden no ser adecuados para tareas que requieren capturar relaciones complejas

Algoritmos no lineales

Ventajas:

Pueden capturar relaciones complejas

Pueden ser más precisos que los algoritmos lineales

Desventajas:

Pueden ser más difíciles de entender y de implementar

Occam's Razor

Occam's Razor, también conocido como la navaja de Occam, es un principio filosófico que sugiere que la explicación más simple es generalmente la correcta. Este principio se puede aplicar al aprendizaje automático para seleccionar entre modelos complejos y simples.

Un modelo complejo puede capturar más información sobre los datos que un modelo simple. Sin embargo, un modelo complejo también puede ser más propenso al sobreajuste. El sobreajuste es un problema en el aprendizaje automático que ocurre cuando un modelo se ajusta demasiado a los datos de entrenamiento y no es capaz de generalizar bien a nuevos datos.

En el contexto del aprendizaje automático, Occam's Razor se puede utilizar de la siguiente manera:

Si un modelo simple puede explicar los datos de entrenamiento con un error aceptable, entonces no es necesario utilizar un modelo complejo.

Un modelo complejo solo debe utilizarse si es necesario para explicar los datos de entrenamiento con un error menor que el que se puede obtener con un modelo simple.

Ejemplo:

Supongamos que tenemos un conjunto de datos de imágenes de gatos y perros. Queremos entrenar un modelo que pueda clasificar correctamente las imágenes en estas dos categorías.

Un modelo simple podría ser un clasificador de vecinos más cercanos. Este modelo clasificaría una imagen como gato o perro en función de la distancia entre la imagen y los gatos y perros conocidos.

Un modelo complejo podría ser una red neuronal artificial. Este modelo podría aprender relaciones más complejas entre las características de la imagen y la categoría de la imagen.

Si el modelo de vecinos más cercanos puede clasificar correctamente las imágenes con un error aceptable, entonces no es necesario utilizar una red neuronal artificial. La red neuronal artificial solo debe utilizarse si es necesario para mejorar el rendimiento del modelo.

El teorema del "No free lunch"

El teorema del "No free lunch" es un teorema en aprendizaje automático que establece que no existe un algoritmo de aprendizaje que funcione mejor que todos los demás para todas las tareas. Este teorema sugiere que es importante elegir un algoritmo adecuado para la tarea específica en cuestión.

El teorema del "No free lunch" se basa en la idea de que cada algoritmo tiene sus propias fortalezas y debilidades. Un algoritmo que funciona bien para una tarea puede no funcionar bien para otra tarea.

Demostración del teorema

El teorema del "No free lunch" se puede demostrar de la siguiente manera:

Supongamos que existe un algoritmo de aprendizaje que funciona mejor que todos los demás para todas las tareas. Este algoritmo tendría un rendimiento superior al de cualquier otro algoritmo para cualquier conjunto de datos de entrenamiento.

Ahora, consideremos un conjunto de datos de entrenamiento que está diseñado para ser desfavorable para el algoritmo. Este conjunto de datos podría estar formado por datos que son ruidosos, incompletos o no representativos de los datos reales.

Si el algoritmo funciona mejor que todos los demás algoritmos para cualquier conjunto de datos de entrenamiento, entonces también debería funcionar mejor que todos los demás algoritmos para este conjunto de datos desfavorable. Sin embargo, esto es imposible, ya que el conjunto de datos está diseñado para ser desfavorable para el algoritmo.

Por lo tanto, no puede existir un algoritmo de aprendizaje que funcione mejor que todos los demás para todas las tareas.

Consecuencias del teorema

El teorema del "No free lunch" tiene varias consecuencias importantes para el aprendizaje automático:

Es importante elegir un algoritmo adecuado para la tarea específica en cuestión. No existe un algoritmo que funcione mejor que todos los demás para todas las tareas.

Es importante considerar las fortalezas y debilidades de cada algoritmo antes de elegir uno. Un algoritmo que funciona bien para una tarea puede no funcionar bien para otra tarea.

Es importante realizar pruebas con diferentes algoritmos para encontrar el que funcione mejor para una tarea específica.